arXiv:1712.00080v2 [cs.CV] 13 Jul 2018

Super SloMo: High Quality Estimation of Multiple Intermediate Frames for Video Interpolation

Huaizu Jiang[1] Deqing Sun[2] Varun Jampani[2]
Ming-Hsuan Yang[3,2] Erik Learned-Miller[1] Jan Kautz[2]
[1]UMass Amherst [2]NVIDIA [3]UC Merced
{hzjiang,elm}@cs.umass.edu,{deqings,vjampani,jkautz}@nvidia.com, mhyang@ucmerced.edu

Abstract

Given two consecutive frames, video interpolation aims at generating intermediate frame(s) to form both spatially and temporally coherent video sequences. While most existing methods focus on single-frame *interpolation, we propose an end-to-end convolutional neural network for* variable-length multi-frame *video interpolation, where the motion interpretation and occlusion reasoning are jointly modeled. We start by computing bi-directional optical flow between the input images using a U-Net architecture. These flows are then linearly combined at each time step to approximate the intermediate bi-directional optical flows. These approximate flows, however, only work well in locally smooth regions and produce artifacts around motion boundaries. To address this shortcoming, we employ another U-Net to refine the approximated flow and also predict soft visibility maps. Finally, the two input images are warped and linearly fused to form each intermediate frame. By applying the visibility maps to the warped images before fusion, we exclude the contribution of occluded pixels to the interpolated intermediate frame to avoid artifacts. Since none of our learned network parameters are time-dependent, our approach is able to produce as many intermediate frames as needed. To train our network, we use 1,132 240-fps video clips, containing 300K individual video frames. Experimental results on several datasets, predicting different numbers of interpolated frames, demonstrate that our approach performs consistently better than existing methods.*

1. Introduction

There are many memorable moments in your life that you might want to record with a camera in *slow-motion* because they are hard to see clearly with your eyes: the first time a baby walks, a difficult skateboard trick, a dog catching a ball, *etc*. While it is possible to take 240-fps (frame-per-second) videos with a cell phone, professional high-speed cameras are still required for higher frame rates. In addition, many of the moments we would like to slow down are unpredictable, and as a result, are recorded at standard frame rates. Recording everything at high frame rates is impractical–it requires large memories and is power-intensive for mobile devices.

Thus it is of great interest to generate high-quality slow-motion video from existing videos. In addition to transforming standard videos to higher frame rates, video interpolation can be used to generate smooth view transitions. It also has intriguing new applications in self-supervised learning, serving as a supervisory signal to learn optical flow from unlabeled videos [15, 16].

It is challenging to generate multiple intermediate video frames because the frames have to be coherent, both spatially and temporally. For instance, generating 240-fps videos from standard sequences (30-fps) requires interpolating seven intermediate frames for every two consecutive frames. A successful solution has to not only correctly interpret the motion between two input images (implicitly or explicitly), but also understand occlusions. Otherwise, it may result in severe artifacts in the interpolated frames, especially around motion boundaries.

Existing methods mainly focus on *single-frame* video interpolation and have achieved impressive performance for this problem setup [15, 16, 19, 20]. However, these methods cannot be directly used to generate arbitrary higher frame-rate videos. While it is an appealing idea to apply a single-frame video interpolation method recursively to generate multiple intermediate frames, this approach has at least two limitations. First, recursive single-frame interpolation cannot be fully parallelized, and is therefore slow, since some frames cannot be computed until other frames are finished (e.g., in seven-frame interpolation, frame 2 depends on 0 and 4, while frame 4 depends on 0 and 8). Errors also accumulates during recursive interpolation. Second, it can only generate $2^i - 1$ intermediate frames (*e.g.*, 3, 7). As a result, one cannot use this approach (efficiently) to generate 1008-fps video from 24-fps, which requires generating 41

intermediate frames.

In this paper we present a high-quality *variable-length multi-frame* interpolation method that can interpolate a frame at any arbitrary time step between two frames. Our main idea is to warp the input two images to the specific time step and then adaptively fuse the two warped images to generate the intermediate image, where the motion interpretation and occlusion reasoning are modeled in a single end-to-end trainable network. Specifically, we first use a *flow computation* CNN to estimate the bi-directional optical flow between the two input images, which is then linearly fused to approximate the required intermediate optical flow in order to warp input images. This approximation works well in smooth regions but poorly around motion boundaries. We therefore use another *flow interpolation* CNN to refine the flow approximations and predict soft visibility maps. By applying the visibility maps to the warped images before fusion, we exclude the contribution of occluded pixels to the interpolated intermediate frame, reducing artifacts. The parameters of both our flow computation and interpolation networks are independent of the specific time step being interpolated, which is an input to the flow interpolation network. Thus, our approach can generate as many intermediate frames as needed in parallel,

Occlusion means blocking objects.

To train our network, we collect 240-fps videos from YouTube and hand-held cameras [29]. In total, we have 1.1K video clips, consisting of 300K individual video frames with a typical resolution of 1080×720. We then evaluate our trained model on several other independent datasets that require different numbers of interpolations, including the Middlebury [1], UCF101 [28], slowflow dataset [10], and high-frame-rate MPI Sintel [10]. Experimental results demonstrate that our approach significantly outperforms existing methods on all datasets. We also evaluate our unsupervised (self-supervised) optical flow results on the KITTI 2012 optical flow benchmark [6] and obtain better results than the recent method [15].

2. Related Work

Video interpolation. The classical approach to video interpolation is based on optical flow [7, 2], and interpolation accuracy is often used to evaluate optical flow algorithms [1, 32]. Such approaches can generate intermediate frames at arbitrary times between two input frames. Our experiments show that state-of-the-art optical flow method [9], coupled with occlusion reasoning [1], can serve as a strong baseline for frame interpolation. However, motion boundaries and severe occlusions are still challenging to existing flow methods [4, 6], and thus the interpolated frames tend to have artifacts around boundaries of moving objects. Furthermore, the intermediate flow computation (*i.e.*, flow interpolation) and occlusion reasoning are based on heuristics and not end-to-end trainable.

Mahajan *et al*. [17] move the image gradients to a given time step and solve a Poisson equation to reconstruct the interpolated frame. This method can also generate multiple intermediate frames, but is computationally expensive because of the complex optimization problems. Meyer *et al*. [18] propose propagating phase information across oriented multi-scale pyramid levels for video interpolation. While achieving impressive performance, this method still tends to fail for high-frequency contents with large motions.

The success of deep learning in high-level vision tasks has inspired numerous deep models for low-level vision tasks, including frame interpolation. Long *et al*. [16] use frame interpolation as a supervision signal to learn CNN models for optical flow. However, their main target is optical flow and the interpolated frames tend to be blurry. Niklaus *et al*. [19] consider the frame interpolation as a local convolution over the two input frames and use a CNN to learn a spatially-adaptive convolution kernel for each pixel. Their method obtains high-quality results. However, it is both computationally expensive and memory intensive to predict a kernel for every pixel. Niklaus *et al*. [20] improve the efficiency by predicting separable kernels. But the motion that can be handled is limited by the kernel size (up to 51 pixels). Liu *et al*. [15] develop a CNN model for frame interpolation that has an explicit sub-network for motion estimation. Their method obtains not only good interpolation results but also promising unsupervised flow estimation results on KITTI 2012. However, as discussed previously, these CNN-based single-frame interpolation methods [19, 20, 15] are not well-suited for multi-frame interpolation.

Wang *et al*. [33] investigate to generate intermediate frames for a light field video using video frames taken from another standard camera as references. In contrast, our method aims at producing intermediate frames for a plain video and does not need reference images.

Learning optical flow. State-of-the-art optical flow methods [35, 36] adopt the variational approach introduce by Horn and Schunck [8]. Feature matching is often adopted to deal with small and fast-moving objects [3, 23]. However, this approach requires the optimization of a complex objective function and is often computationally expensive. Learning is often limited to a few parameters [13, 26, 30].

Recently, CNN-based models are becoming increasingly popular for learning optical flow between input images. Dosovitskiy *et al*. [5] develop two network architectures, FlowNetS and FlowNetC, and show the feasibility of learning the mapping from two input images to optical flow using CNN models. Ilg *et al*. [9] further use the FlowNetS and FlowNetC as building blocks to design a larger network, FlowNet2, to achieve much better performance. Two recent methods have also been proposed [22, 31] to build the classical principles of optical flow into the network architecture,

achieving comparable or even better results and requiring less computation than FlowNet2 [9].

In addition to the supervised setting, learning optical flow using CNNs in an unsupervised way has also been explored. The main idea is to use the predicted flow to warp one of the input images to another. The reconstruction error serves as a supervision signal to train the network. Instead of merely considering two frames [38], a memory module is proposed to keep the temporal information of a video sequence [21]. Similar to our work, Liang *et al*. [14] train optical flow via video frame extrapolation, but their training uses the flow estimated by the EpicFlow method [23] as an additional supervision signal.

3. Proposed Approach

In this section, we first introduce optical flow-based intermediate frame synthesis in section 3.1. We then explain details of our flow computation and flow interpolation networks in section 3.2. In section 3.3, we define the loss function used to train our networks.

3.1. Intermediate Frame Synthesis

Given two input images I_0 and I_1 and a time $t \in (0, 1)$, our goal is to predict the intermediate image $\hat{I}_t$ at time $T = t$. A straightforward way is to accomplish this is to train a neural network [16] to directly output the RGB pixels of $\hat{I}_t$. In order to do this, however, the network has to learn to interpret not only the motion pattens but also the appearance of the two input images. Due to the rich RGB color space, it is hard to generate high-quality intermediate images in this way. Inspired by [1] and recent advances in single intermediate video frame interpolation [19, 20, 15], we propose fusing the warped input images at time $T = t$.

Let $F_{t\to0}$ and $F_{t\to1}$ denote the optical flow from I_t to I_0 and I_t to I_1, respectively. If these two flow fields are known, we can synthesize the intermediate image $\hat{I}_t$ as follows:

$$\hat{I}_t = \alpha_0 \odot g(I_0, F_{t\to0}) + (1 - \alpha_0) \odot g(I_1, F_{t\to1}), \quad (1)$$

where $g(\cdot, \cdot)$ is a *backward warping* function, which can be implemented using bilinear interpolation [40, 15] and is differentiable. The parameter α_0 controls the contribution of the two input images and depend on two factors: temporal consistency and occlusion reasoning. $\odot$ denotes element-wise multiplication, implying content-aware weighting of input images. For temporal consistency, the closer the time step $T = t$ is to $T = 0$, the more contribution I_0 makes to $\hat{I}_t$; a similar property holds for I_1. On the other hand, an important property of the video frame interpolation problem is that if a pixel p is visible at $T = t$, it is most likely *at least visible in one of the input images*,[1] which means

[1] It is a rare case but it may happen that an object appears and disappears between I_0 and I_1.

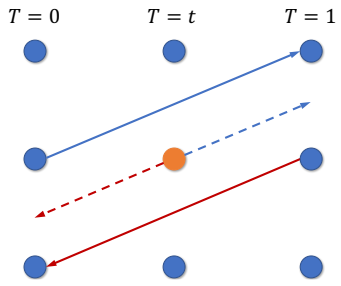


Figure 1: Illustration of intermediate optical flow approximation. The orange pixel borrows optical flow from pixels at the same position in the first and second images.

the occlusion problem can be addressed. We therefore introduce *visibility maps* $V_{t\leftarrow0}$ and $V_{t\leftarrow1}$. $V_{t\leftarrow0}(p) \in [0, 1]$ denotes whether the pixel p remains visible (0 is fully occluded) when moving from $T = 0$ to $T = t$. Combining the temporal consistency and occlusion reasoning, we have

$$\hat{I}_t = \frac{1}{Z} \odot \big((1-t)V_{t\leftarrow0} \odot g(I_0, F_{t\to0}) + tV_{t\leftarrow1} \odot g(I_1, F_{t\to1})\big),$$

where $Z = (1 - t)V_{t\to0} + tV_{t\to1}$ is a normalization factor.

3.2. Arbitrary-time Flow Interpolation

Since we have no access to the target intermediate image I_t, it is hard to compute the flow fields $F_{t\to0}$ and $F_{t\to1}$. To address this issue, we can approximately synthesize the intermediate optical flow $F_{t\to0}$ and $F_{t\to1}$ using the optical flow between the two input images $F_{0\to1}$ and $F_{1\to0}$.

Consider the toy example shown in Fig. 1, where each column corresponds to a certain time step and each dot represents a pixel. For the orange dot p at $T = t$, we are interested in synthesizing its optical flow to its corresponding pixel at $T = 1$ (the blue dashed arrow). One simple way is to borrow the optical flow *from the same grid positions* at $T = 0$ and $T = 1$ (blue and red solid arrows), assuming that the optical flow field is locally smooth. Specifically, $F_{t\to1}(p)$ can be approximated as

$$\hat{F}_{t\to1}(p) = (1 - t)F_{0\to1}(p) \quad (2)$$

or

$$\hat{F}_{t\to1}(p) = -(1 - t)F_{1\to0}(p), \quad (3)$$

where we take the direction of the optical flow between the two input images in the same or opposite directions and scale the magnitude accordingly ($(1 - t)$ in (3)). Similar to the temporal consistency for RGB image synthesis, we can approximate the intermediate optical flow by combining the bi-directional input optical flow as follows (in vector form).

$$\begin{aligned} \hat{F}_{t\to0} &= -(1 - t)tF_{0\to1} + t^2F_{1\to0} \\ \hat{F}_{t\to1} &= (1 - t)^2F_{0\to1} - t(1 - t)F_{1\to0}. \end{aligned} \quad (4)$$

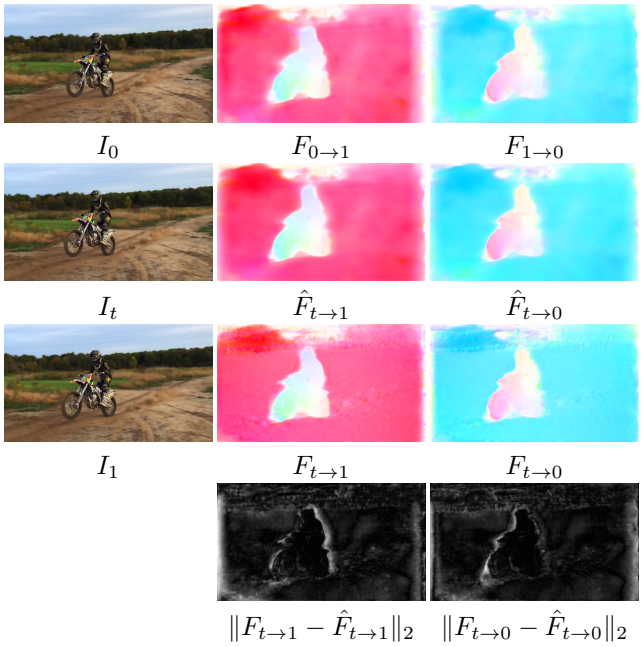


Figure 2: Samples of flow interpolation results, where $t = 0.5$. The entire scene is moving toward the left (due to camera translation) and the motorcyclist is independently moving left. The last row shows that the refinement from our flow interpolation CNN is mainly around the motion boundaries (the whiter a pixel, the bigger the refinement).

This approximation works well in smooth regions but poorly around motion boundaries, because the motion near motion boundaries is not locally smooth. To reduce artifacts around motion boundaries, which may cause poor image synthesis, we propose learning to refine the initial approximation. Inspired by the cascaded architecture for optical flow estimation in [9], we train a flow interpolation sub-network. This sub-network takes the input images I_0 and I_1, the optical flows between them $F_{0\to1}$ and $F_{0\to1}$, the flow approximations $\hat{F}_{t\to0}$ and $\hat{F}_{0\to1}$, and two warped input images using the approximated flows $g(I_0, \hat{F}_{t\to0})$ and $g(I_1, \hat{F}_{t\to1})$ as input, and outputs refined intermediate optical flow fields $F_{t\to1}$ and $F_{t\to0}$. Sample interpolation results are displayed in Figure 2.

As discussed in Section 3.1, visibility maps are essential to handle occlusions. Thus, We also predict two visibility maps $V_{t\leftarrow0}$ and $V_{t\leftarrow1}$ using the flow interpolation CNN, and enforce them to satisfy the following constraint

$$V_{t\leftarrow0} = 1 - V_{t\leftarrow1}. \tag{5}$$

Without such a constraint, the network training diverges. Intuitively, $V_{t\leftarrow0}(p) = 0$ implies $V_{t\leftarrow1}(p) = 1$, meaning that the pixel p from I_0 is occluded at $T = t$, we should fully trust I_1 and vice versa. Note that it rarely happens that a pixel at time t is occluded both at time 0 and 1. Since

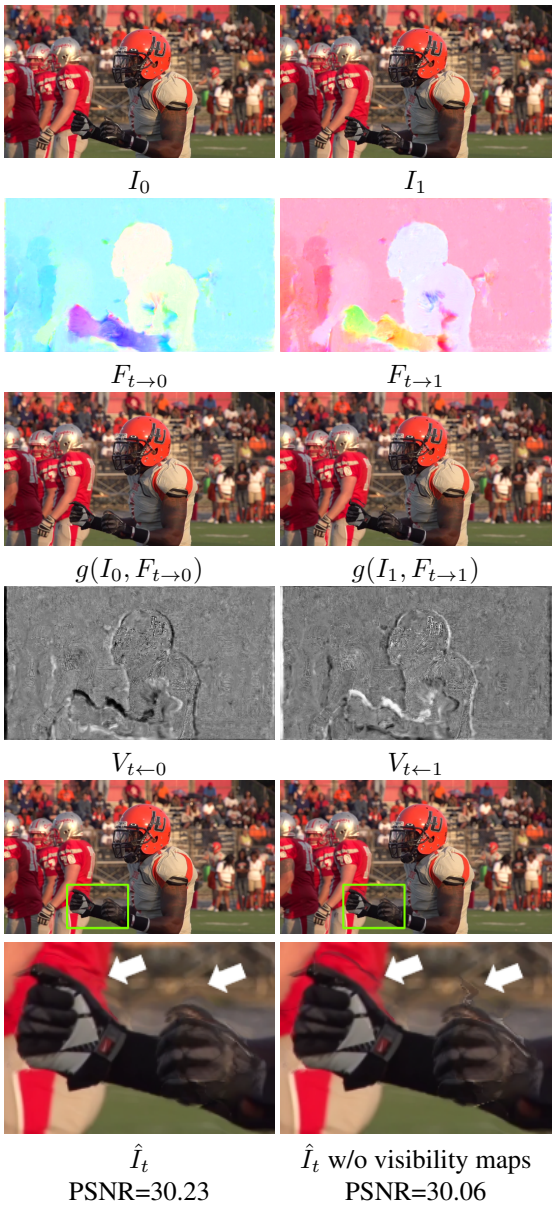


Figure 3: Samples of predicted visibility maps (best viewed in color), where $t = 0.5$. The arms move downwards from $T = 0$ to $T = 1$. So the area right above the arm at $T = 0$ is visible at t but the area right above the arm at $T = 1$ is occluded (*i.e.*, invisible) at t. The visibility maps in the fourth row clearly show this phenomenon. The white area around arms in $V_{t\leftarrow0}$ indicate such pixels in I_0 contribute most to the synthesized $\hat{I}_t$ while the occluded pixels in I_1 have little contribution. Similar phenomena also happen around motion boundaries (*e.g.*, around bodies of the athletes).

we use soft visibility maps, when the pixel p is visible both in I_0 and I_1, the network learns to adaptively combine the information from two images, similarly to the matting effect [24]. Samples of learned visibility maps are shown in

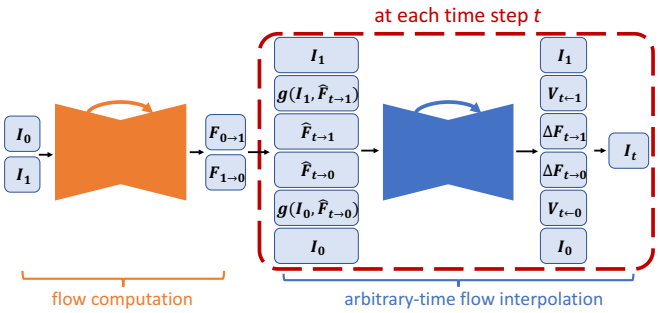


Figure 4: Network architecture of our approach.

the fourth row of Fig. 3.

In order to do flow interpolation, we need to first compute the bi-directional optical flow between the two input images. Recent advances in deep learning for optical flow have demonstrated great potential to leverage deep CNNs to reliably estimate optical flow. In this paper, we train a flow computation CNN, taking two input images I_0 and I_1, to jointly predict the forward optical flow $F_{0\to1}$ and backward optical flow $F_{1\to0}$ between them.

Our entire network is summarized in Fig. 4. For the flow computation and flow interpolation CNNs, we adopt the U-Net architecture [25]. The U-Net is a fully convolutional neural network, consisting of an encoder and a decoder, with skip connections between the encoder and decoder features at the same spatial resolution For both networks, we have 6 hierarchies in the encoder, consisting of two convolutional and one Leaky ReLU ($\alpha = 0.1$) layers. At the end of each hierarchy except the last one, an average pooling layer with a stride of 2 is used to decrease the spatial dimension. There are 5 hierarchies in the decoder part. At the beginning of each hierarchy, a bilinear upsampling layer is used to increase the spatial dimension by a factor of 2, followed by two convolutional and Leaky ReLU layers.

For the flow computation CNN, it is crucial to have large filters in the first few layers of the encoder to capture long-range motion. We therefore use 7×7 kernels in the first two convoluional layers and 5×5 in the second hierarchy. For layers in the rest of entire network, we use 3×3 convolutional kernels. The detailed configuration of the network is described in our supplementary material.

We found concatenating output of the encoders in two networks together as input to the decoder of the flow interpolation network yields slightly better results. Moreover, instead of directly predicting the intermediate optical flow in the flow interpolation network, we found it performs slightly better to predict intermediate optical flow residuals. In specific, the flow interpolation network predicts $\Delta F_{t\to0}$ and $\Delta F_{t\to1}$. We then have

$$\begin{aligned} F_{t\to0} &= \hat{F}_{t\to0} + \Delta F_{t\to0} \\ F_{t\to1} &= \hat{F}_{t\to1} + \Delta F_{t\to1} \end{aligned} \tag{6}$$

3.3. Training

Given input images I_0 and I_1, a set of intermediate frames $\{I_{t_i}\}_{i=1}^N$ between them, where $t_i \in (0, 1)$, and our predictions of intermediate frames $\{\hat{I}_{t_i}\}_{i=1}^N$, our loss function is a linear combination of four terms:

$$l = \lambda_r l_r + \lambda_p l_p + \lambda_w l_w + \lambda_s l_s. \tag{7}$$

Reconstruction loss l_r models how good the reconstruction of the intermediate frames is:

$$l_r = \frac{1}{N} \sum_{i=1}^{N} \|\hat{I}_{t_i} - I_{t_i}\|_1. \tag{8}$$

Such a reconstruction loss is defined in the RGB space, where pixel values are in the range $[0, 255]$.

Perceptual loss. Even though we use the L_1 loss to model the reconstruction error of intermediate frames, it might still cause blur in the predictions. We therefore use a perceptual loss [11] to preserve details of the predictions and make interpolated frames sharper, similar to [20]. Specifically, the perceptual loss l_p is defined as

$$l_p = \frac{1}{N} \sum_{i=1}^{N} \|\phi(\hat{I}_t) - \phi(I_t)\|_2, \tag{9}$$

where ϕ denote the `conv4_3` features of an ImageNet pre-trained VGG16 model [27]

Warping loss. Besides intermediate predictions, we also introduce the warping loss l_w to model the quality of the computed optical flow, defined as

$$\begin{aligned} l_w =& \|I_0 - g(I_1, F_{0\to1})\|_1 + \|I_1 - g(I_0, F_{1\to0})\|_1 + \\ & \frac{1}{N} \sum_{i=1}^{N} \|I_{t_i} - g(I_0, \hat{F}_{t_i\to0})\|_1 + \frac{1}{N} \sum_{i=1}^{N} \|I_{t_i} - g(I_1, \hat{F}_{t_i\to1})\|_1. \end{aligned} \tag{10}$$

Smoothness loss. Finally, we add a smoothness term [15] to encourage neighboring pixels to have similar flow values:

$$l_s = \|\nabla F_{0\to1}\|_1 + \|\nabla F_{1\to0}\|_1. \tag{11}$$

The weights have been set empirically using a validation set as $\lambda_r = 0.8$, $\lambda_p = 0.005$, $\lambda_w = 0.4$, and $\lambda_s = 1$. Every component of our network is differentiable, including warping and flow approximation. Thus our model can be end-to-end trained.

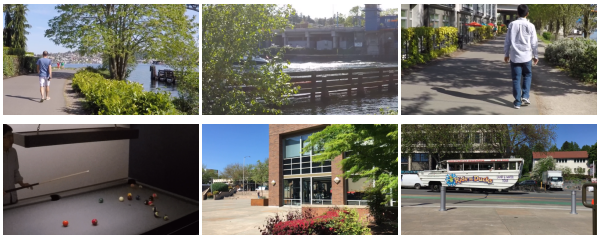
4. Experiments

4.1. Dataset

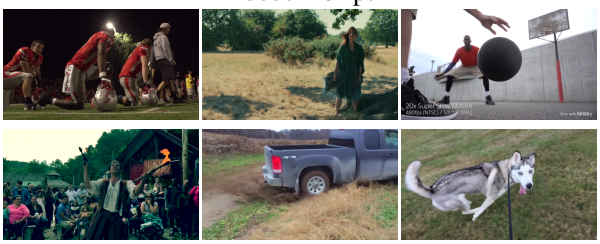
To train our network, we use the 240-fps videos from [29], taken with hand-held cameras. We also collect a dataset of 240-fps videos from YouTube. Table 1 summarizes the statistics of the two datasets and Fig. 5 shows

Table 1: Statistics of dataset we use to train our network.

	Adobe240-fps [29]	YouTube240-fps
#video clips	118	1,014
#video frames	79,768	296,352
mean #frames per clip	670.3	293.1
resolution	720p	720p



Adobe240-fps



YouTube240-fps

Figure 5: Snapshot of our training data.

a snapshot of randomly sampled video frames. In total, we have 1,132 video clips and 376K individual video frames. There are a great variety of scenes in both datasets, from indoor to outdoor, from static to moving cameras, from daily activities to professional sports, etc.

We train our network using all of our data and test our model on several independent datasets, including the Middlebury benchmark [1], UCF101 [28], slowflow dataset [10], and high-frame-rate Sintel sequences [10]. For Middlebury, we submit our single-frame video interpolation results of eight sequences to its evaluation server. For UCF101, in every triple of frames, the first and third ones are used as input to predict the second frame using 379 sequences provided by [15]. The slowflow dataset contains 46 videos taken with professional high-speed cameras. We use the first and eighth video frames as input, and interpolate intermediate 7 frames, equivalent to converting a 30-fps video to a 240-fps one. The original Sintel sequences [4] were rendered at 24 fps. 13 of them were re-rendered at 1008 fps [10]. To convert from 24-fps to 1008-fps using a video frame interpolation approach, one needs to insert 41 in-between frames. However, as discussed in the introduction, it is not directly possible with recursive single-frame interpolation methods [19, 20, 15] to do so. Therefore, we instead predict 31 in-between frames for fair comparisons with previous methods.

Table 2: Effectiveness of multi-frame video interpolation on the *Adobe240-fps* dataset.

	PSNR	SSIM	IE
1 interp	30.26	0.909	8.85
3 interp	31.02	0.917	8.43
7 interp	**31.19**	**0.918**	**8.30**

Table 3: Effectiveness of different components of our model on the *Adobe240-fps* dataset.

	PSNR	SSIM	IE
w/o flow interpolation	30.34	0.908	8.93
w/o vis map	31.16	0.918	8.33
w/o perceptual loss	30.96	0.916	8.50
w/o warping loss	30.52	0.910	8.80
w/o smoothness loss	**31.19**	**0.918**	**8.26**
full model	**31.19**	**0.918**	8.30

Our network is trained using the Adam optimizer [12] for 500 epochs. The learning rate is initialized to be 0.0001 and decreased by a factor of 10 every 200 epochs. During training, all video clips are first divided into shorter ones with 12 frames in each and there is no overlap between any of two clips. For data augmentation, we randomly reverse the direction of entire sequence and select 9 consecutive frames for training. On the image level, each video frame is resized to have a shorter spatial dimension of 360 and a random crop of 352×352 plus horizontal flip are performed.

For evaluation, we report Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) scores between predictions and ground-truth in-between video frames, as well as the interpolation error (IE) [1], which is defined as root-mean-squared (RMS) difference between the ground-truth image and the interpolated image.

4.2. Ablation Studies

In this section, we perform ablation studies to analyze our model. For the first two experiments, we randomly sampled 107 videos from Adobe240-fps dataset for training and the remaining 12 ones for testing.

Effectiveness of multi-frame video interpolation. We first test whether jointly predicting several in-between frames improves the video interpolation results. Intuitively, predicting a set of in-between frames together might implicitly enforce the network to generate temporally coherent sequences.

To this end, we train three variants of our model: predicting intermediate single, three, and seven frames, which are all evenly distributed across time steps. At test time, we use each model to predict seven in-between frames. Table 2

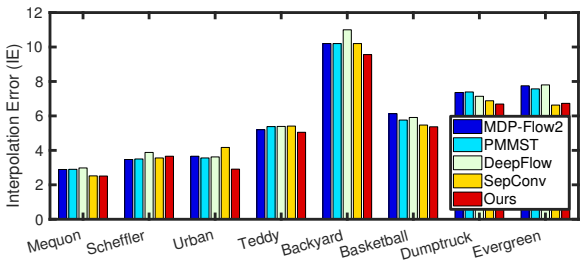


Figure 6: Performance comparisons on each sequence of the Middlebury dataset. Numbers are obtained from the Middlebury evaluation server.

Table 4: Results on the *UCF101* dataset.

	PSNR	SSIM	IE
Phase-Based [18]	32.35	0.924	8.84
FlowNet2 [1, 9]	32.30	0.930	8.40
DVF [15]	32.46	0.930	8.27
SepConv [20]	33.02	0.935	8.03
Ours (Adobe240-fps)	32.84	0.935	8.04
Ours	**33.14**	**0.938**	**7.80**

clearly demonstrates that the more intermediate frames we predict during training, the better the model is.

Impact of different components design. We also investigate the contribution of each component in our model. In particular, we study the impact of flow interpolation by removing the flow refinement from the second U-Net (but keep using the visibility maps). We further study the use of visibility maps as means of occlusion reasoning. We can observe from Table 3 that removing each of three components harms performance. Particularly, the flow interpolation plays a crucial role, which verifies our motivation to introduce the second learned network to refine intermediate optical flow approximations. Adding visibility maps improves the interpolation performance slightly. Without it, there are artifacts generated around motion boundaries, as shown in Figure 3. Both of these validate our hypothesis that jointly learning motion interpretation and occlusion reasoning helps video interpolation.

We also study different loss terms, where the warping loss is the most important one. Although adding the smoothness terms slightly hurts the performance quantitatively, we fount it is useful to generate visually appealing optical flow between input images.

Impact of the number of training samples. Finally, we investigate the effect of the number of training samples. We compare two models: one trained on the Adobe240-fps dataset only and the other one trained on our full dataset. The performance of these two models on the UCF101 dataset can be found in last two rows Table 4. We can see that our model benefits from more training data.

Table 5: Results on the *slowflow* dataset.

	PSNR	SSIM	IE
Phase-Based [18]	31.05	0.858	8.21
FlowNet2 [1, 9]	34.06	**0.924**	**5.35**
SepConv [20]	32.69	0.893	6.79
Ours	**34.19**	**0.924**	6.14

Table 6: Results on the high-frame-rate *Sintel* dataset.

	PSNR	SSIM	IE
Phase-Based [18]	28.67	0.840	10.24
FlowNet2 [1, 9]	30.79	0.922	5.78
SepConv [20]	31.51	0.911	6.61
Ours	**32.38**	**0.927**	**5.42**

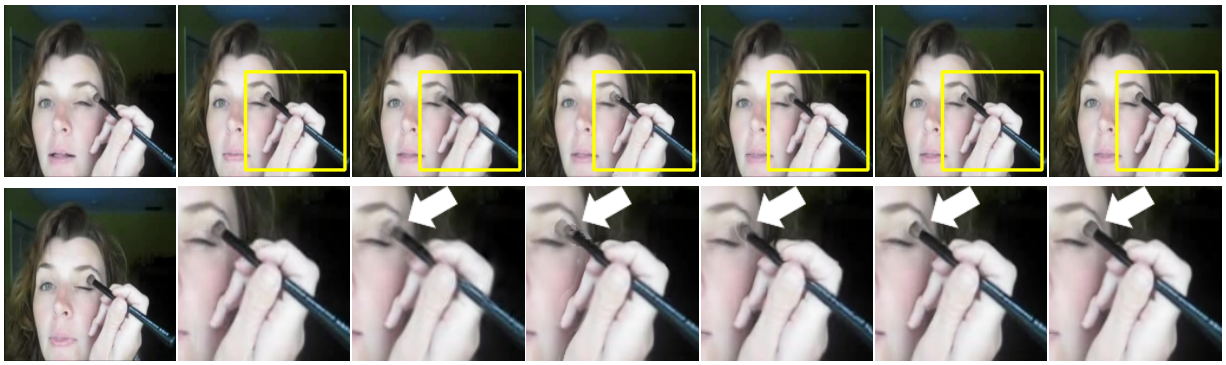
4.3. Comparison with state-of-the-art methods

In this section, we compare our approach with state-of-the-art methods including phase-based interpolation [18], separable adaptive convolution (SepConv) [20], and deep voxel flow (DVF) [15]. We also implement a baseline approach using the interpolation algorithm presented in [1], where we use FlowNet2 [9] to compute the bi-directional optical flow results between two input images. FlowNet2 is good at capturing global background motion and recovering sharp motion boundaries for the optical flow. Thus, when coupled with occlusion reasoning [1], FlowNet2 serves as a strong baseline.

Single-frame video interpolation. The interpolation error (IE) scores on each sequence form the Middlebury dataset are shown in Figure 6. In addition to SepConv, we also compare our model with other three top-performing models on the Middlebury dataset[2], where the interpolation algorithm [1] is coupled with different optical flow methods including MDP-Flow2 [37], PMMST [39], and DeepFlow [34]. Our model achieves the best performance on 6 out of all 8 sequences. Particularly, the `Urban` sequence is generated synthetically and the `Teddy` sequence contains actually two stereo pairs. The performance of our model validates the generalization ability of our approach.

On UCF101, we compute all metrics using the motion masks provided by [15]. The quantitative results are shown in Table 4, highlighting the performance of each interpolation model's capacity to deal with challenging motion regions. Our model consistently outperforms both non-neural [18] and CNN-based approaches [20, 15]. Sam-

[2] http://vision.middlebury.edu/flow/eval/results/results-i1.php



2 input images | actual in-between | PhaseBased [18] | FlowNet2 [1, 9] | DVF [15] | SepConv [20] | ours

Figure 7: Visual results of a sample from UCF101. Our model produces less artifacts around the brush and the hand (best viewed in color). Please see the supplementary material for more image and *video* results.

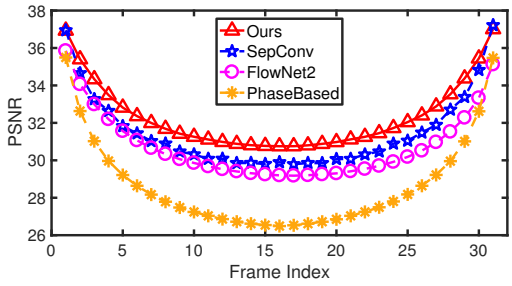


Figure 8: PSNR at each time step when generating 31 intermediate frames on the high-frame-rate Sintel dataset.

ple interpolation results on a sample from UCF101 can be found at Figure 7. More results can be found in our supplementary material.

Multi-frame video interpolation. For the slowflow dataset, we predict 7 in-between frames. All experiments are performed on the half-resolution images with a spatial dimension of 1280×1024. On this dataset, our approach achieves the best PSNR and SSIM scores and FlowNet2 achieves the best SSIM and L_1 error scores. FlowNet2 is good at capturing global motions and thus produces sharp prediction results on those background regions, which follow a global motion pattern. Detailed visual comparisons can be found in our supplementary material.

On the challenging high-frame-rate Sintel dataset, our approach significantly outperforms all other methods. We also show the PSNR scores at each time step in Figure 8. Our approach produces the best predictions for each in-between time step except slightly worse than SepConv at the last time step.

In summary, our approach achieves state-of-the-art results over all datasets, generating single or multiple intermediate frames. It is remarkable, considering the fact our model can be directly applied to different scenarios without any modification.

Table 7: Optical flow results on the KITTI 2012 benchmark.

	LDOF[3]	EpicFlow[23]	FlowNetS[5]	DVF[15]	Ours
EPE	12.4	3.8	9.1	14.6	13.0

4.4. Unsupervised Optical Flow

Our video frame interpolation approach has an unsupervised (self-supervised) network (the flow computation CNN) that can compute the bi-directional optical flow between two input images. Following [15], we evaluate our unsupervised forward optical flow results on the testing set of KITTI 2012 optical flow benchmark [6]. The average end-point error (EPE) scores of different methods are reported in Table 7. Compared with previous unsupervised method DVF [15], our model achieves an average EPE of 13.0, an 11% relative improvement. Very likely this improvement results from the multi-frame video interpolation setting, as DVF [15] has a similar U-Net architecture to ours.

5. Conclusion

We have proposed an end-to-end trainable CNN that can produce as many intermediate video frames as needed between two input images. We first use a flow computation CNN to estimate the bidirectional optical flow between the two input frames, and the two flow fields are linearly fused to approximate the intermediate optical flow fields. We then use a flow interpolation CNN to refine the approximated flow fields and predict soft visibility maps for interpolation. We use more than 1.1K 240-fps video clips to train our network to predict seven intermediate frames. Ablation studies on separate validation sets demonstrate the benefit of flow interpolation and visibility map. Our multi-frame approach consistently outperforms state-of-the-art single frame methods on the Middlebury, UCF101, slowflow, and high-frame-rate Sintel datasets. For the unsupervised learning of optical

flow, our network outperforms the recent DVF method [15] on the KITTI 2012 benchmark.

Acknowledgement

We would like to thank Oliver Wang for generously sharing the Adobe 240-fps data [29]. Yang acknowledges support from NSF (Grant No. 1149783).

A. Network Architecture

Our flow computation and flow interpolation CNNs share a similar U-Net architecture, shown in Figure 9.

B. Visual Comparisons on UCF101

Figure 10 and Figure 11 show visual comparisons of single-frame interpolation results on the UCF101 dataset. For more visual comparisons, please refer to our supplementary video http://jianghz.me/projects/superslomo/superslomo_public.mp4.

References

[1] S. Baker, D. Scharstein, J. P. Lewis, S. Roth, M. J. Black, and R. Szeliski. A database and evaluation methodology for optical flow. *IJCV*, 92(1):1–31, 2011. 2, 3, 6, 7, 8

[2] J. Barron, D. Fleet, and S. Beauchemin. Performance of optical flow techniques. *IJCV*, 12(1):43–77, 1994. 2

[3] T. Brox and J. Malik. Large displacement optical flow: Descriptor matching in variational motion estimation. *PAMI*, 33(3):500–513, 2011. 2, 8

[4] D. J. Butler, J. Wulff, G. B. Stanley, and M. J. Black. A naturalistic open source movie for optical flow evaluation. In *ECCV*, 2012. 2, 6

[5] A. Dosovitskiy, P. Fischery, E. Ilg, C. Hazirbas, V. Golkov, P. van der Smagt, D. Cremers, T. Brox, et al. Flownet: Learning optical flow with convolutional networks. In *ICCV*, 2015. 2, 8

[6] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? The KITTI vision benchmark suite. In *CVPR*, 2012. 2, 8

[7] E. Herbst, S. Seitz, and S. Baker. Occlusion reasoning for temporal interpolation using optical flow. Technical report, August 2009. 2, 11, 12

[8] B. Horn and B. Schunck. Determining optical flow. *Artificial Intelligence*, 16:185–203, 1981. 2

[9] E. Ilg, N. Mayer, T. Saikia, M. Keuper, A. Dosovitskiy, and T. Brox. Flownet 2.0: Evolution of optical flow estimation with deep networks. In *CVPR*, 2017. 2, 3, 4, 7, 8, 11, 12

[10] J. Janai, F. Güney, J. Wulff, M. Black, and A. Geiger. Slow flow: Exploiting high-speed cameras for accurate and diverse optical flow reference data. In *CVPR*, 2017. 2, 6

[11] J. Johnson, A. Alahi, and L. Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *ECCV*, 2016. 5

[12] D. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 6

[13] Y. Li and D. P. Huttenlocher. Learning for optical flow using stochastic optimization. In *ECCV*, 2008. 2

[14] X. Liang, L. Lee, W. Dai, and E. P. Xing. Dual motion GAN for future-flow embedded video prediction. In *ICCV*, 2017. 3

[15] Z. Liu, R. Yeh, X. Tang, Y. Liu, and A. Agarwala. Video frame synthesis using deep voxel flow. In *ICCV*, 2017. 1, 2, 3, 5, 6, 7, 8, 9, 11, 12

[16] G. Long, L. Kneip, J. M. Alvarez, H. Li, X. Zhang, and Q. Yu. Learning image matching by simply watching video. In *ECCV*, 2016. 1, 2, 3

[17] D. Mahajan, F.-C. Huang, W. Matusik, R. Ramamoorthi, and P. Belhumeur. Moving gradients: a path-based method for plausible image interpolation. *ACM TOG*, 28(3):42, 2009. 2

[18] S. Meyer, O. Wang, H. Zimmer, M. Grosse, and A. Sorkine-Hornung. Phase-based frame interpolation for video. In *CVPR*, 2015. 2, 7, 8, 11, 12

[19] S. Niklaus, L. Mai, and F. Liu. Video frame interpolation via adaptive convolution. In *CVPR*, 2017. 1, 2, 3, 6

[20] S. Niklaus, L. Mai, and F. Liu. Video frame interpolation via adaptive separable convolution. In *ICCV*, 2017. 1, 2, 3, 5, 6, 7, 8, 11, 12

[21] V. Patraucean, A. Handa, and R. Cipolla. Spatio-temporal video autoencoder with differentiable memory. In *ICLR, workshop*, 2016. 3

[22] A. Ranjan and M. J. Black. Optical flow estimation using a spatial pyramid network. In *CVPR*, 2017. 2

[23] J. Revaud, P. Weinzaepfel, Z. Harchaoui, and C. Schmid. EpicFlow: Edge-Preserving Interpolation of Correspondences for Optical Flow. In *CVPR*, 2015. 2, 3, 8

[24] C. Rhemann, C. Rother, J. Wang, M. Gelautz, P. Kohli, and P. Rott. A perceptually motivated online benchmark for image matting. In *CVPR*, 2009. 4

[25] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015. 5

[26] S. Roth and M. J. Black. On the spatial statistics of optical flow. *IJCV*, 74(1):33–50, 2007. 2

[27] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *CoRR*, abs/1409.1556, 2014. 5

[28] K. Soomro, A. R. Zamir, and M. Shah. Ucf101: A dataset of 101 human action classes from videos in the wild. *CRCV-TR-12-01*, 2012. 2, 6

[29] S. Su, M. Delbracio, J. Wang, G. Sapiro, W. Heidrich, and O. Wang. Deep video deblurring. In *CVPR*, 2017. 2, 5, 6, 9

[30] D. Sun, S. Roth, J. P. Lewis, and M. J. Black. Learning optical flow. In *ECCV*, 2008. 2

[31] D. Sun, X. Yang, M.-Y. Liu, and J. Kautz. Pwc-net: Cnns for optical flow using pyramid, warping, and cost volume. In *CVPR*, 2018. 2

[32] R. Szeliski. Prediction error as a quality metric for motion and stereo. In *ICCV*, 1999. 2

[33] T. Wang, J. Zhu, N. K. Kalantari, A. A. Efros, and R. Ramamoorthi. Light field video capture using a learning-based hybrid imaging system. *ACM TOG*, 36(4). 2

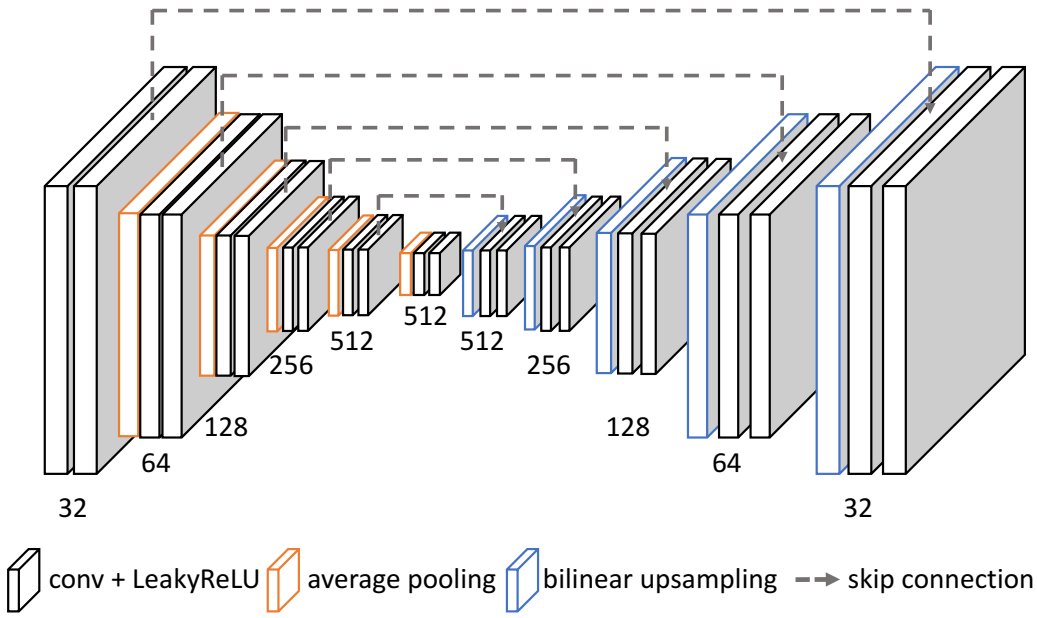


Figure 9: Illustration of the architecture of our flow computation and flow interpolation CNNs.

[34] P. Weinzaepfel, J. Revaud, Z. Harchaoui, and C. Schmid. Deepflow: Large displacement optical flow with deep matching. In *ICCV*, 2013. 7

[35] J. Wulff, L. Sevilla-Lara, and M. J. Black. Optical flow in mostly rigid scenes. In *CVPR*, 2017. 2

[36] J. Xu, R. Ranftl, and V. Koltun. Accurate optical flow via direct cost volume processing. In *CVPR*, 2017. 2

[37] L. Xu, J. Jia, and Y. Matsushita. Motion detail preserving optical flow estimation. *TPAMI*, 34(9):1744–1757, 2012. 7

[38] J. J. Yu, A. W. Harley, and K. G. Derpanis. Back to basics: Unsupervised learning of optical flow via brightness constancy and motion smoothness. In *ECCV, workshop*, 2016. 3

[39] F. Zhang, S. Xu, and X. Zhang. High accuracy correspondence field estimation via mst based patch matching. 2015. 7

[40] T. Zhou, S. Tulsiani, W. Sun, J. Malik, and A. A. Efros. View synthesis by appearance flow. In *ECCV*, 2016. 3

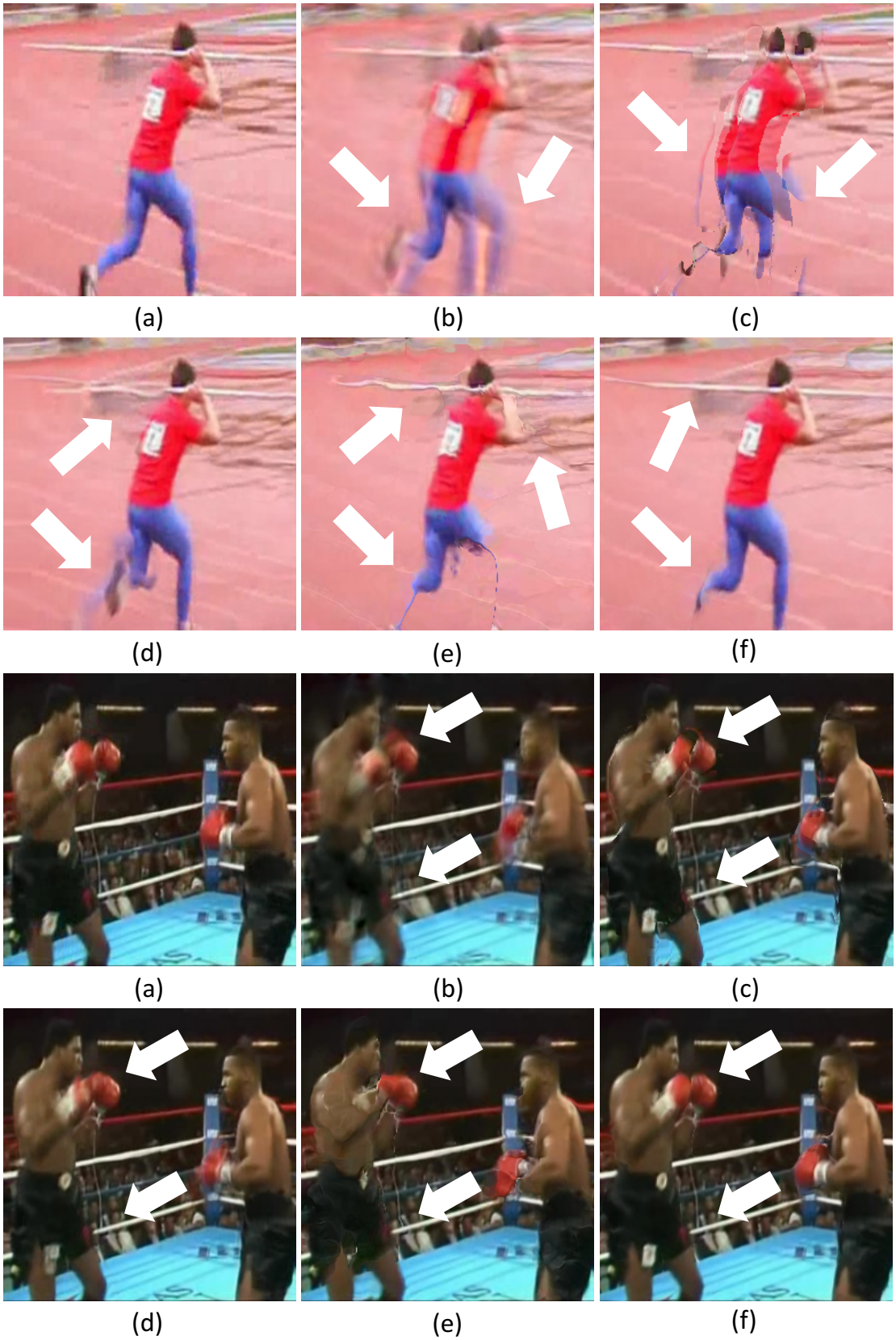


Figure 10: Visual comparisons on the UCF101 dataset. (a) actual in-between frame, interpolation results from (b) Phase-Based [18], (c) FlowNet2 [7, 9], (d) SepConv [20], (e) DVF [15], and (f) Ours.

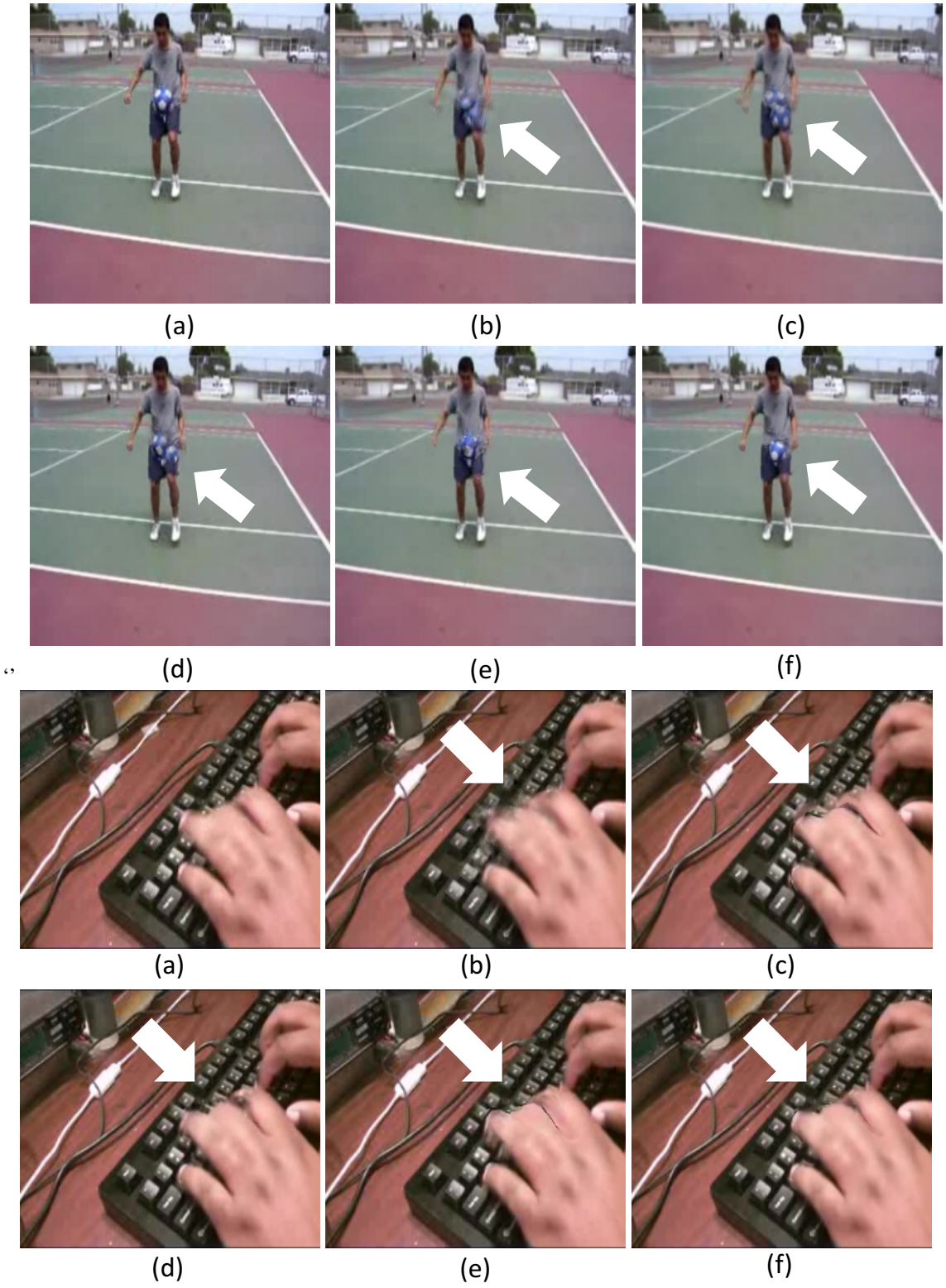


Figure 11: Visual comparisons on the UCF101 dataset. (a) actual in-between frame, interpolation results from (b) Phase-Based [18], (c) FlowNet2 [7, 9], (d) SepConv [20], (e) DVF [15], and (f) Ours.